

Stroke Prediction Project

1. Introduction.....	1
2. Dataset Description	1
3. Exploratory Data Analysis (EDA).....	2
4. Data Preprocessing	2
5. Machine Learning Model	2
6. Model Evaluation.....	3
7. Application Interface	3
8. System Architecture	3
9. Limitations	4
10. Conclusion.....	4

1. Introduction

The purpose of this project is to build a machine learning system that predicts stroke risk based on patient health data. The project emphasizes a structured and production-like workflow, where data processing, model training, and inference are tightly aligned.

A key objective is not only to train a model, but to ensure consistency between training and real-world usage through a unified pipeline. This reduces common errors such as mismatched data formats between training and the user interface.

The project addresses challenges such as missing values, categorical encoding, and class imbalance, while focusing on building a reliable and reusable system.

2. Dataset Description

The project uses a publicly available dataset containing patient health and demographic information. Each row represents one individual, and each column represents a feature.

Key variables include age, gender, marital status, work type, residence type, glucose level, BMI, smoking status, and the target variable `stroke`, which indicates whether the individual has experienced a stroke.

The dataset contains both numerical and categorical features and includes missing values in the BMI column. It is also class-imbalanced, with significantly fewer stroke cases than non-stroke cases.

3. Exploratory Data Analysis (EDA)

EDA was performed to understand data distribution and quality.

Key findings:

- Significant class imbalance in the target variable
- Missing values in BMI
- Numerical features vary widely in scale
- Categorical features require encoding

The analysis guided the design of the preprocessing pipeline.

4. Data Preprocessing

Preprocessing is implemented using a scikit-learn Pipeline and ColumnTransformer, ensuring consistency between training and inference.

Numerical features:

- Imputation using median
- Standard scaling

Categorical features:

- Imputation using most frequent value
- One-hot encoding

Binary features:

- Passed through without transformation to preserve their meaning

All preprocessing steps are combined into a single pipeline, which is later reused by the model and the application.

This is a key improvement over the previous version, as it eliminates discrepancies between training data and user input.

5. Machine Learning Model

A Logistic Regression model is used for binary classification.

Instead of training the model separately, it is now integrated into a full pipeline:

Pipeline structure:

Preprocessing → Logistic Regression

This ensures:

- The model always receives correctly transformed data
- No manual preprocessing is required during inference

Logistic regression is chosen for:

- Simplicity

- Interpretability
- Strong baseline performance.

6. Model Evaluation

The model was evaluated using a test dataset that was not used during training.

Accuracy: ~ 0.75

Metric	No Stroke (0)	Stroke (1)
Precision	0.99	0.14
Recall	0.74	0.80
F1-score	0.85	0.23

Interpretation:

- The model performs well on the majority class (no stroke)
- High recall for stroke cases means most true stroke cases are detected
- Low precision indicates many false positives

This trade-off is expected due to class imbalance and reflects a bias toward detecting stroke cases.

7. Application Interface

A user interface is implemented using **Streamlit**.

Key improvements:

- The UI sends raw input directly to the trained pipeline
- No manual preprocessing is performed in the UI
- The pipeline handles all transformations internally

This resolves previous issues where:

- Input data format did not match training data
- Manual transformations caused runtime errors

The result is a more robust and maintainable application.

8. System Architecture

The project follows a modular and production-oriented structure:

- Training module: builds and saves the pipeline
- Preprocessing logic: embedded inside the pipeline
- Model artifact: saved and reused
- Application layer: loads pipeline and performs predictions

Workflow:

Raw Data → Pipeline (Preprocessing + Model) → Evaluation → Save Model → Streamlit UI → Prediction

Key architectural improvement:

- A single source of truth (pipeline) is used for both training and inference

This eliminates inconsistencies and simplifies the system.

9. Limitations

- The dataset is relatively small
- Strong class imbalance affects precision
- No advanced techniques (e.g., SMOTE, ensemble models) are applied
- The model is not clinically validated

Additionally:

- Logistic regression may be too simple to capture complex patterns.

10. Conclusion

This project demonstrates a complete and improved machine learning workflow with a strong focus on correctness and consistency.

The main advancement compared to the earlier version is the introduction of a unified pipeline, which:

- Aligns training and inference
- Reduces errors
- Improves maintainability

The final system is a functional application that reflects real-world machine learning practices, particularly in how data preprocessing and model prediction are tightly integrated.